Table of Contents

myAGV — ArUco Camera Detection & Robotic Arm Control (General Guide)

A general, reusable guide for two common tasks on the myAGV platform:

1. **ArUco marker detection** using the front (CSI) camera
2. **Controlling the robotic arm** (MechArm 270)

This is written for building your **own** applications — it is not tied to any particular demo. The hardware facts (camera pipeline, calibration, arm connection, pump) apply to any project on this robot.

Part 1 — ArUco Marker Detection (front camera)

1.1 Camera at a glance

Item	Value
Sensor	IMX219 CSI camera (front)
Access method	GStreamer pipeline via `nvarguscamerasrc` (Jetson)
Typical working resolution	960 × 540
ArUco library	OpenCV `cv2.aruco`
Common dictionary	`DICT_6X6_250` (pick any; both ends must match)

> **Why GStreamer, not `cv2.VideoCapture(0)`?** The CSI camera on the Jetson is not a normal `/dev/video` webcam — it must be opened through the `nvarguscamerasrc` pipeline. A plain `VideoCapture(0)` will fail or return black frames. (A USB webcam, if you add one, *can* use the normal `VideoCapture(index)`.)

1.2 Opening the camera

```
import cv2, cv2.aruco as aruco, numpy as np

def gstreamer_pipeline(sensor_id=0, capture_width=3264, capture_height=
2464,
                       display_width=960, display_height=540,
                       framerate=21, flip_method=0):
    return (
        "nvarguscamerasrc sensor-id=%d !"
        "video/x-raw(memory:NVMM), width=(int)%d, height=(int)%d, frame
```

```
rate=(fraction)%d/1 ! "
        "nvvidconv flip-method=%d ! "
        "video/x-raw, width=(int)%d, height=(int)%d, format=(string)BGR
x ! "
        "videoconvert ! video/x-raw, format=(string)BGR ! "
        "appsink drop=True max-buffers=1 emit-signals=True"
        % (sensor_id, capture_width, capture_height, framerate,
           flip_method, display_width, display_height))

cam = cv2.VideoCapture(gstreamer_pipeline(flip_method=0), cv2.CAP_GSTRE
AMER)
assert cam.isOpened(), "Camera failed to open"
```

`flip_method` (0–7) rotates/mirrors the stream in hardware. Set it once so the image comes out upright for your mounting instead of flipping every frame in software.

1.3 Camera calibration (intrinsics)

Pose estimation (distance/angle) needs the camera's intrinsic matrix and distortion coefficients. Below is a set measured for this myAGV's CSI camera at 960×540 — a good starting point, but **re-calibrate for your own unit** if you need accurate distances (each camera/lens differs slightly):

```
camera_matrix = np.array([[785.855437,   0.0,       451.670922],
                          [  0.0,       584.820336, 259.056856],
                          [  0.0,         0.0,        1.0     ]])

dist_coeffs = np.array([[0.095135, -0.109279, -0.002513, -0.002418, 0.0
]])
```

To calibrate your own camera, use the standard OpenCV chessboard procedure (or ROS `camera_calibration`). The matrix is resolution-specific — recompute it if you change the capture resolution.

1.4 Detecting markers in a frame

```
ret, frame = cam.read()
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

aruco_dict = cv2.aruco.getPredefinedDictionary(aruco.DICT_6X6_250)
parameters = cv2.aruco.DetectorParameters()

corners, ids, _ = aruco.detectMarkers(gray, aruco_dict, parameters=para
meters)

if ids is not None:
    frame = aruco.drawDetectedMarkers(frame.copy(), corners, ids)  # op
tional overlay
    print("Detected IDs:", ids.flatten())
```

Reliability tip: `gray = cv2.equalizeHist(gray)` before `detectMarkers` boosts contrast and improves the detection rate in uneven lighting.

1.5 Getting 3-D pose (distance + orientation)

```
marker_length = 0.05   # metres — the REAL printed side length of your marker

rvec, tvec, _ = cv2.aruco.estimatePoseSingleMarkers(
    corners, marker_length, camera_matrix, dist_coeffs)

# tvec[i][0] = [X, Y, Z] in metres, in the camera frame
x_cm = tvec[0][0][0] * 100
y_cm = tvec[0][0][1] * 100
z_cm = tvec[0][0][2] * 100     # Z = straight-line distance to the marker
print(f"Marker distance Z = {z_cm:.1f} cm")

R, _ = cv2.Rodrigues(rvec[0][0])   # 3x3 rotation matrix for orientation
```

What the tvec numbers mean

Value	Meaning	Typical use
`X`	left/right offset of marker vs image centre	lateral alignment (strafe)
`Y`	up/down offset	height checks
`Z`	distance from camera to marker	approach / stop distance

⚠️ **`marker_length` must equal the real printed size.** If it's wrong, `Z` scales wrong by the same ratio — the distance is only as accurate as this number and the calibration.

1.6 Locating a marker in the image (pixel position)

Independent of 3-D pose, you often just need where a marker sits in the frame — e.g. to centre the robot on it:

```
c  = corners[i][0]                                  # 4 corners of marker i
cx = (c[0][0] + c[1][0] + c[2][0] + c[3][0]) / 4    # centre X (pixels)
cy = (c[0][1] + c[1][1] + c[2][1] + c[3][1]) / 4    # centre Y (pixels)
perc = cx / 960.0                                   # 0.5 = horizontally centred
```

Drive/strafe the robot until `perc ≈ 0.5` to centre on the marker; use `cy` (or Z) to judge how close you are.

1.7 Handling multiple markers

`ids` and `corners` are parallel arrays — iterate them together and branch on the ID:

```
if ids is not None:
    for i, mid in enumerate(ids.flatten()):
        c  = corners[i][0]
        cx = (c[0][0]+c[1][0]+c[2][0]+c[3][0]) / 4
        print(f"ID {mid} at x={cx:.0f}")
```

Keep the ID numbers you care about in a small config/constant at the top of your script so they're easy to change — don't scatter magic numbers through the code.

1.8 Quick checklist / common issues

Symptom	Likely cause
Black frames / camera won't open	Used `VideoCapture(0)` instead of the GStreamer pipeline
Marker never detected	Dictionary mismatch (printed vs code), or marker too small/blurry
Distance (Z) is off	Wrong `marker_length`, or camera not calibrated for this resolution
Image upside-down / mirrored	Set the correct `flip_method` (or `cv2.flip`)
Intermittent detection	Add `cv2.equalizeHist`; improve lighting; increase marker size

---

Part 2 — Robotic Arm Control (MechArm 270)

2.1 Arm at a glance

Item	Value
Arm model	MechArm 270 (`pymycobot.mecharm270.MechArm270`)
Connection	USB serial, `/dev/ttyACM0` @ `115200` baud
Joints	6 (J1–J6), angles in **degrees**
Coordinate mode	X, Y, Z in mm + RX, RY, RZ in degrees
End effector	depends on setup (e.g. suction pump, gripper)

> Using a different arm (e.g. myCobot 280)? The API below is the same across `pymycobot` arms — just import the matching class (e.g. `MyCobot280`) and use its port. Joint counts and reach differ, so poses are not interchangeable.

2.2 Connecting to the arm

```
from pymycobot.mecharm270 import MechArm270

mc = MechArm270('/dev/ttyACM0', 115200)
```

> If the arm doesn't respond: confirm the port exists (`ls /dev/ttyACM*`), that no other program holds it, and that your user has serial permission (`dialout` group).

2.3 The two ways to move the arm

A. Joint angles — `send_angles(angles, speed)` Set the 6 joints directly (degrees). Most repeatable way to reach a "named pose".

```
# [J1, J2, J3, J4, J5, J6],  speed 0–100
mc.send_angles([0, 0, 0, 0, 0, 0], 50)   # e.g. zero position
```

B. Cartesian coordinates — `send_coords(coords, speed, mode)` Move the tool tip to an X/Y/Z position (mm) with orientation. Best for straight-line moves like "lift 5 cm on Z".

```
# [X, Y, Z, RX, RY, RZ],  speed 0–100,  mode: 0=free path, 1=straight l
ine
mc.send_coords([150, -60, 220, -175, 0, -90], 40, mode=1)
```

Reading the current position

```
angles = mc.get_angles()   # -> [J1..J6] degrees, or None if busy
coords = mc.get_coords()   # -> [X,Y,Z,RX,RY,RZ], or None if busy
```

> `get_coords()` / `get_angles()` can return `None` or `-1` while the arm is moving. Always loop until you get a valid reading before using it:

```
curr = mc.get_coords()
while curr is None:
    time.sleep(0.5)
    curr = mc.get_coords()
```

2.4 Named poses (recommended pattern)

Instead of scattering raw angles through your code, keep a dictionary of named poses and call them by name. Teach a pose once, reuse it everywhere:

```
poses = {
    "home":   [0, 0, 0, 0, 0, 0],
    "ready":  [90, -30, 22, -1, 87, 0],
    # add your own...
}

mc.send_angles(poses["ready"], 50)
```

To teach / record a pose:

```
# 1. Move the arm (by hand in free-move, or with jog controls) to the p
osition
# 2. Read the angles back and paste the 6 numbers into your table:
print(mc.get_angles())
```

2.5 End effector (example: suction pump)

The end effector is switched separately from arm motion. On this myAGV the suction pump is driven through the Jetson's GPIO; the project code wraps it as helper calls:

```
pump_on()     # engage (grab)
time.sleep(1.0)   # allow vacuum/grip to build
# ... move ...
pump_off()    # release
```

> If you use a gripper instead, `pymycobot` provides gripper commands (`set_gripper_state`, `set_gripper_value`, …). The pattern is the same: **grab → move → release**.

2.6 A generic pick-and-place sequence

Angles to approach → engage → lift → move → place → release:

```
import time

# 1. Move above the object, then to the grab pose
mc.send_angles(poses["ready"], 50);  time.sleep(2)
mc.send_angles(poses["grab"],  50);  time.sleep(2)

# 2. Engage end effector
pump_on();  time.sleep(1.0)

# 3. Lift straight up 5 cm (linear move on Z)
curr = mc.get_coords()
while curr is None:
    time.sleep(0.5); curr = mc.get_coords()
curr[2] += 50                       # Z + 50 mm
mc.send_coords(curr, 40, mode=1)

# 4. Carry to the drop location
mc.send_angles(poses["carry"], 60)
mc.send_angles(poses["place"], 60)

# 5. Lower, release, retreat
curr = mc.get_coords()
while curr is None:
    time.sleep(0.5); curr = mc.get_coords()
```

```
curr[2] -= 45                       # Z - 45 mm
mc.send_coords(curr, 40, mode=1)
pump_off();  time.sleep(0.5)
curr[2] += 45
mc.send_coords(curr, 40, mode=1)
mc.send_angles(poses["home"], 50)
```

API reference: OpenCV `cv2.aruco` for detection; `pymycobot` (`MechArm270`) for arm control. Camera access uses the Jetson `nvarguscamerasrc` GStreamer pipeline.